

Information Flow Control and Parametrized Monads

Jack Liell-Cock

Draft of 04-06-2026

This note outlines a comparison between a coarse-grained (CG) information-flow control calculus [9, 10] and parametrized monads [3]. The claim we develop is that the CG type constructor $\text{CG } \ell_i \ell_o \tau$ is a parametrized monad over the security lattice viewed as a thin category. We show the correspondence and find that the CG constructor has structure beyond the parametrized monad. Namely, the type **Labeled**, the operations **unlabel** and **toLabeled**, and the invariant $\ell_i \sqsubseteq \ell_o$. This additional data is captured by a *representability isomorphism* $\text{CG } \ell_i \ell_o \tau \cong \text{CG } \ell_i \ell_i (\text{Labeled } \ell_o \tau)$, which corresponds to the decomposition underlying the fine-grained/coarse-grained translation [10]. We contrast this against other results in the literature [8, 5].

1 Objects of comparison

The CG calculus. The coarse-grained calculus [10] (the static fragment of HLIO [4], itself built on LIO [11]) has a monadic type $\text{CG } \ell_i \ell_o \tau$, where ℓ_i and ℓ_o range over a security lattice $(\mathcal{L}, \sqsubseteq, \sqcup, \perp, \top)$. Informally, ℓ_i is the program counter on entry and ℓ_o on exit. The program counter accumulates the security levels the computation reads, and lower-bounds what it can write. Some of the defining rules are:

$$\begin{array}{c}
 \text{RET} \\
 \frac{e : \tau}{\text{ret}(e) : \text{CG } \ell \ell \tau} \\
 \\
 \text{UNLABEL} \\
 \frac{e : \text{Labeled } \ell \tau}{\text{unlabel}(e) : \text{CG } \ell_i (\ell_i \sqcup \ell) \tau} \\
 \\
 \text{BIND} \\
 \frac{e_1 : \text{CG } \ell_i \ell \tau \quad x : \tau \vdash e_2 : \text{CG } \ell \ell_o \tau'}{\text{bind}(e_1, x.e_2) : \text{CG } \ell_i \ell_o \tau'} \\
 \\
 \text{TO LABELED} \\
 \frac{e : \text{CG } \ell_i \ell_o \tau}{\text{toLabeled}(e) : \text{CG } \ell_i \ell_i (\text{Labeled } \ell_o \tau)}
 \end{array}$$

with subtyping contravariant in ℓ_i and covariant in ℓ_o , and the standing invariant $\ell_i \sqsubseteq \ell_o$.

Parametrized monads. A parametrized monad [3] on a category $\mathcal{C}$ indexed by a category $\mathcal{S}$ is a functor $T : \mathcal{S}^{\text{op}} \times \mathcal{S} \times \mathcal{C} \rightarrow \mathcal{C}$ with a unit $\eta_A : A \rightarrow T(S, S, A)$, dinatural in S , and a multiplication

$$\mu : T(S_1, S_2, T(S_2, S_3, A)) \longrightarrow T(S_1, S_3, A),$$

dinatural in S_2 and natural in S_1 and S_3 , subject to unit and associativity laws. Kleisli composition is defined when the output index of one computation matches the input index of the next. Atkey gives the corresponding typed command calculus with judgement $\Gamma; S_1 \vdash^c c : A; S_2$, and the algebraic theory [2].

2 The correspondence

Let $\mathcal{S} := (\mathcal{L}, \sqsubseteq)$ be a thin category. Then $\text{CG } \ell_i \ell_o \tau$ is a parametrized monad as summarised by Table 1. Everything above the divider is parametrized-monad structure. Below is further structure which we elaborate in Section 3.

As $\mathcal{S}$ is thin, there is at most one arrow $S \rightarrow S'$ (namely $S \sqsubseteq S'$); so dinaturality diagrams for η and μ therefore collapse to inequalities, which are the subtyping coercions [10].

CG construct	Parametrized-monad structure
CG $\ell_i \ell_o \tau$	$T(\ell_i, \ell_o, \tau)$
ret	$\eta_A : A \rightarrow T(S, S, A)$
bind	Kleisli composition
subtyping	functoriality in $\mathcal{S}^{\text{op}} \times \mathcal{S}$
subtyping coercion	dinaturality of η and μ
bind with effects (ref, deref, assign)	strength
invariant $\ell_i \sqsubseteq \ell_o$	none
Labeled $\ell \tau$	none
unlabel	none
toLabeled	none

Table 1: CG as a parametrized monad.

3 The extra structure

Three features of CG are structure beyond that of a parametrized monad.

1. CG requires $\ell_i \sqsubseteq \ell_o$ as a well-formedness side condition, restricting CG $\ell_i \ell_o \tau$ to the upper-triangle of $\mathcal{S}^{\text{op}} \times \mathcal{S}$.
2. Labeled ℓ is an $\mathcal{L}$ -graded monad, unlabel is the graded action that raises the ambient post-index from ℓ_i to $\ell_i \sqcup \ell$, and the family $\{\text{Labeled } \ell\}_\ell$ is the family-of-monads of DCC [1].
3. toLabeled takes CG $\ell_i \ell_o \tau$ to CG $\ell_i \ell_i (\text{Labeled } \ell_o \tau)$. It freezes the output security level into a returned Labeled ℓ_o value while resetting the computation's output security level to ℓ_i .

Conjecture 3.1. *For all $\ell_i \sqsubseteq \ell_o$ and τ ,*

$$\text{CG } \ell_i \ell_o \tau \cong \text{CG } \ell_i \ell_i (\text{Labeled } \ell_o \tau)$$

naturally in τ and monotone in ℓ_i and ℓ_o .

Since $\sqcup$ is idempotent, CG $\ell \ell$ is a monad, and is moreover idempotent [5, §9.1], i.e. re-running at a fixed security level does nothing. Conjecture 3.1 is the two-index (parametrized) analogue, which factors the CG constructor through the diagonal idempotent monad and the graded Labeled layer. The forward map is toLabeled and the backwards map is $\text{bind}(-, y.\text{unlabel}(y))$. The backward map type-checks by BIND/UNLABEL because the premise $\ell_i \sqsubseteq \ell_o$ implies $\ell_i \sqcup \ell_o = \ell_o$.

The outstanding obligation is the round-trip toLabeled/unlabel identities, which hold morally for LIO and should follow from the monad laws together with idempotency of $\sqcup$.

4 The coarse/fine translation

The decomposition Conjecture 3.1 is the one that relates the FG and CG systems in [10]. In FG every value type has the form A^ℓ (a base type carrying an intrinsic label) and computations are typed under a program counter pc . The type translation $\llbracket - \rrbracket$ from FG to CG [10] sets

$$\llbracket A^\ell \rrbracket = \text{Labeled } \ell \llbracket A \rrbracket, \quad \llbracket \Sigma; \Psi; \Gamma \vdash_{pc} e : \tau \rrbracket = \Sigma; \Psi; \llbracket \Gamma \rrbracket \vdash \llbracket e \rrbracket : \forall \alpha \sqsubseteq pc. \text{CG } \alpha \alpha \llbracket \tau \rrbracket.$$

That is, fine-grained per-value labels become the Labeled layer, and fine-grained computations become diagonal CG computations CG $\alpha \alpha$, polymorphic in the program counter α . The translation target is therefore exactly the right-hand side of Conjecture 3.1, CG $\alpha \alpha (\text{Labeled } \ell -)$, packaged under a quantifier over the fibre α .

The coercion is the isomorphism witness. In the case where the FG translation goes through ($\alpha \sqsubseteq \ell$, so $\llbracket \tau \rrbracket = \text{Labeled } \ell' \llbracket A \rrbracket$ for some $\ell' \sqsupseteq \ell$ [10]), coerce $\text{CG } \alpha \ell \llbracket \tau \rrbracket$ to $\text{CG } \alpha \alpha \llbracket \tau \rrbracket$ with

$$\text{toLabeled}(\text{bind}(-, y.\text{unlabel}(y))),$$

which is the composite of the two maps in Conjecture 3.1.

An obstruction. The full FG to CG translation fails (forcing the fragment FG^- [10]) in the case $\alpha \not\sqsubseteq \ell$. Here, one must coerce $\text{CG } \alpha \ell (\text{Labeled } \ell' -)$ down to $\text{CG } \alpha \alpha (\text{Labeled } \ell' -)$, but `toLabeled` only produces

$$\text{CG } \alpha \alpha (\text{Labeled } \ell (\text{Labeled } \ell' -)).$$

This is a nested label that cannot be collapsed when ℓ and ℓ' are incomparable. The isomorphism produces $\text{CG } \alpha \alpha (\text{Labeled } \ell -)$ faithfully, but it cannot merge the stacked labels at incomparable levels. Hence the isomorphism explains the shape of the translation, but does not resolve the obstruction forcing FG^- .

5 Other literature

Graded vs. parametrized. Category-graded monads (lax functors $\mathcal{S} \rightarrow \text{Endo}(\mathcal{C})^{\text{op}}$) subsume both graded and parametrized monads [8] where the Kleisli category is the Grothendieck construction. CG is a category-graded monad over $\mathcal{L}$ together with the generalised unit of [8, §6.2, Prop. 37] (the right lax natural transformation that supplies the approximation/subtyping coercions). [8] also classify LIO-style IFC tracking as parametrized.

Collapsing parametrized monads to graded monads. Any parametrized monad over a monoidal index $(\mathcal{S}, \sqcup, \perp)$ can be collapsed to a graded monad via an end [8, §8.1, Prop. 41], provided $\mathcal{C}$ is finitely complete. For CG , this yields the graded monad $\int_{\ell} \text{CG } \ell (\ell \sqcup -) (=)$. This collapse is lossy in that it quotients the input index away. Conjecture 3.1 alleviates this lossiness with a CG -specific, information-preserving *isomorphism* that exploits idempotency rather than completeness.

Denotational semantics of IFC. Categorical models and noninterference (via a *presence-absence* technique) for DCC, an IFC calculus, are established [5]. [1] gives the original model. But this does not address the parametrized layer. The nearest relatives of Conjecture 3.1 are the *absorption* isomorphism $S_{\ell}A \cong A$ when $\ell \sqsubseteq A$ [5, Lem. 9.1] and the characterisation of the DCC protection judgement as carrying an Eilenberg-Moore algebra [5, Thm. 9.6]. Neither is the reification isomorphism for the parametrized constructor. [10] already give a semantics for CG , but via logical relations. The closest existing semantics is the monadic account of program-counter labels [7], but this is for a noninterference proof technique.

What is novel. Denotational semantics of IFC is not novel [5]. Neither is IFC as a parametrized/graded monad [8], `Labeled` as graded monad [1], or as a coeffect graded comonad [6].

The identification of $\text{CG } \ell_i \ell_o \tau$ as a parametrized monad over $\mathcal{L}$ is not in the literature but likely folklore. Conjecture 3.1, enabled by idempotency of $\sqcup$ is new. It is sharper than [8, Prop. 41] (lossy) and beyond the single-index formulation [5, §9].

The two-layer reading of CG as an outer parametrized monad whose Kleisli values inhabit an inner graded `Labeled`, glued by Conjecture 3.1, is also new. This is a categorification of the syntactic CG/FG translation [10]. It validates the shape of the translation, but not its totality. The nested-label obstruction restricting the source to FG^- is orthogonal to the monad/`Labeled` factorisation.

We could also aim for a denotational noninterference proof for CG in the two-index setting, extending the presence-absence technique of [5] to the parametrized setting.

Remark. The strong-monoidal collapse of [5, §7.2] requires the graded structure maps to be invertible, which holds in the lattice/idempotent setting but not for parametrized monads in general. This indicates Conjecture 3.1 is genuinely lattice-specific rather than an instance of a general theorem.

References

- [1] Martín Abadi, Anindya Banerjee, Nevin Heintze, and Jon G. Riecke. A core calculus of dependency. In *Proc. POPL*, 1999. doi:10.1145/292540.292555.
- [2] Robert Atkey. Algebras for parameterised monads. In *Proc. CALCO*, 2009. doi:10.1007/978-3-642-03741-2_2.
- [3] Robert Atkey. Parameterised notions of computation. *Journal of Functional Programming*, 2009. doi:10.1017/S095679680900728X.
- [4] Pablo Buiras, Dimitrios Vytiniotis, and Alejandro Russo. HLIO: Mixing static and dynamic typing for information-flow control in Haskell. In *Proc. ICFP*, 2015. doi:10.1145/2784731.2784758.
- [5] Pritam Choudhury. Monadic and comonadic aspects of dependency analysis. *Proc. OOPSLA*, 2022. doi:10.1145/3563335.
- [6] Marco Gaboardi, Shin-ya Katsumata, Dominic Orchard, Flavien Breuvar, and Tarmo Uustalu. Combining effects and coeffects via grading. In *Proc. ICFP*, 2016. doi:10.1145/2951913.2951939.
- [7] Andrew K. Hirsch and Ethan Cecchetti. Giving semantics to program-counter labels via secure effects. *Proc. POPL*, 2021. doi:10.1145/3434316.
- [8] Dominic Orchard, Philip Wadler, and Harley Eades III. Unifying graded and parameterised monads. In *Proc. MSFP*, 2020. doi:10.4204/EPTCS.317.2.
- [9] Vineet Rajani, Iulia Bastys, Willard Rafnsson, and Deepak Garg. Type systems for information flow control: The question of granularity. *ACM SIGLOG News*, 2017. doi:10.1145/3051528.3051531.
- [10] Vineet Rajani and Deepak Garg. On the expressiveness and semantics of information flow types. *Journal of Computer Security*, 2020. doi:10.3233/JCS-191382.
- [11] Deian Stefan, Alejandro Russo, John C. Mitchell, and David Mazières. Flexible dynamic information flow control in Haskell. In *Haskell Symposium*, 2011. doi:10.1145/2034675.2034688.